

Hooks

Overview

In this section you will create hooks that automatically run quality checks and maintenance tasks for your Flappy Kiro game development workflow.

Hooks

[Agent Hooks](#) are powerful automation tools that streamline your development workflow by automatically executing predefined agent actions when specific events occur in your IDE. With hooks, you eliminate the need to manually request routine tasks and ensure consistency across your codebase.

Create New Hook

Create an agent hook

Hooks let you automate agent workflows by triggering actions when specific events occur in your IDE. You can set up hooks to run when you save a file, complete an agent execution, enter a prompt, or manually click a button. Hooks can either send a prompt to the agent or execute shell commands.



1. Event occurs

The hook is triggered when the specified event, such as file creation or user prompt submission, occurs.

2. Actions performed

Instructions are sent to the agent, or a shell command is executed.

3. Context updated

The result of the agent instruction or shell command execution is added to the agent's context.

Title

Document Generator

Description

Updates the README.md file based on the contents of the current source code file.

Event

File Saved

File path(s) to watch

src/**/*.{js,ts,jsx,tsx}

Action

Ask Kiro

Run Command

Provides instructions to the agent.

Instructions for Kiro agent

Generate comprehensive documentation for the current file:
1. Extract function and class signatures
2. Document parameters and return types
3. Provide usage examples based on existing code
4. Update the README.md with any new exports
5. Ensure documentation follows project standards

Clear

Create Hook

Agent Hooks are automated triggers that execute predefined agent actions when specific events occur in your IDE. Rather than manually asking for routine tasks to be performed, hooks set up automated responses to events such as file changes, prompt submissions, tool invocations, task execution, and manual triggers.

Agent Hooks transform your development workflow through intelligent automation. By setting up hooks for common tasks, you can:

- Maintain consistent code quality.
- Prevent security vulnerabilities.
- Reduce manual overhead.
- Standardize team processes.
- Create faster development cycles.

Whether you're working on a small project or managing a large codebase, Agent Hooks help ensure that routine tasks are handled automatically and consistently, allowing you to focus on building great software.

Create Git automation hook

Automatically commit all changes to Git repository whenever Kiro completes its tasks, including all hook executions.

► **Example prompt**

Create code quality hook

Ask Kiro to create a hook that helps enforce code quality.

► **Example prompts**

Create documentation sync hook

Listens to source code changes and automatically updates project documentation in README or docs folder.

► **Example prompt**

(Optional) Create performance monitoring hook

In the chat panel, ask Kiro to create a game performance audit hook.

► **Example prompts**

(Optional) Create game state validation hook

Ask Kiro to create a game logic validation hook.

► **Example prompts**

Test hooks

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. [Privacy policy](#) [Terms of use](#) [Cookie preferences](#)

1. Save a game component → Performance audit runs.
2. Modify game mechanics → Validation hook triggers.
3. Run code quality hook → Standards compliance checked.

Summary

You implemented automated quality assurance hooks that ensure optimal game performance, validate game logic consistency, and maintain code quality standards throughout Flappy Kiro development.

